



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Reporte Técnico

Reconocimiento de figuras

Proyecto I

P R E S E N T A N

Cruz Prieto Donovan Yoab

Gómez Flores Sergio Erick

Navarro Moreno Daniel

Núñez Gutiérrez Johan



Ciudad Universitaria, CDMX. 2026

1. Definición del problema

El propósito general del proyecto era diseñar una solución eficiente y rentable para automatizar el reconocimiento y clasificación de figuras geométricas a través de la ruta de una imagen de tipo .bpm.

Entradas: El programa recibe la ruta de una imagen (.bpm) con las siguientes características:

- El fondo es de un color sólido
- Cada figura tiene un color sólido, distinto entre figuras y distinto del fondo
- No hay anti-aliasing ni colores intermedios en los bordes
- Las figuras no se traslapan entre sí
- Tamaño, posición y rotación de las figuras son arbitrarios.

Salida: Se muestra al usuario una lista ordenada de las métricas de las figuras encontradas en la imagen, así como su clasificación acorde al tipo de figura que sea (cuadrilátero, círculo, triángulo, otros), por ejemplo:

```
=====
Figura 1:
-> Color: #E38225
-> Vértices (RDP)   : 4
-> Área            : 107503.0 px2
-> Perímetro       : 1308.00 px
-> Fact. Circularidad: 0.7896
-> Categoría       : Cuadrilátero (C)
=====
```

2. Arsenal

2.1 Lenguaje Utilizado

Para el desarrollo de este proyecto se utilizó Java como lenguaje de programación por diversos motivos:

- **Orientación a objetos:** Al ser un lenguaje orientado a objetos, resulta bastante útil para la reutilización de código, encapsulamiento de datos internos, rendimiento en el manejo de memoria para estructuras de datos y mantenimiento de proyectos grandes.
- **Permite crear programas modulares:** Java permite dividir un programa en partes más pequeñas, independientes y manejables, llamadas módulos o subprogramas. Así pudimos delegar la carga de trabajo entre todos los integrantes del equipo y permitir que cada quien avanzara simultáneamente sin menguar el progreso de los demás miembros.
- **Familiaridad:** Antes de escribir una sola línea de código, nos organizamos y concluimos que Java es el lenguaje que mejor manejamos, por lo que facilitaría la implementación de métodos complejos que veremos más adelante.

2.2 Bibliotecas elegidas

- **java.awt:** La elegimos porque ayuda a representar coordenadas bidimensionales (Point) y además nos permite almacenar y modificar imágenes directamente en la memoria (BufferedImage).
- **java.io:** Fue elegida porque facilita la representación de forma abstracta de rutas de archivos (file), útil para verificar la existencia de la imagen (.bmp) antes de trabajar con ella.
- **java.util:** Esta biblioteca es el pilar del proyecto, proporciona las estructuras de datos necesarias para su implementación.
- **javax.imageio:** Se eligió porque permite la decodificación y lectura del archivo .bmp.

3. Análisis del problema

3.1 Supuestos

Como se menciona en la sección 1. Definición del problema, se asumen las siguientes condiciones iniciales:

Formato soportado:

- La entrada siempre será un archivo .bmp

Condiciones del archivo .bmp:

- El fondo es de un color sólido.
- Cada figura tiene un color sólido, distinto entre figuras y distinto del fondo,
- No hay anti-aliasing ni colores intermedios en los bordes.
- Las figuras no se traslapan entre sí.
- Tamaño, posición y rotación de las figuras son arbitrarios.

3.2 Casos limite

Figuras en las esquinas de la imagen.

Descripción: Para identificar el color de fondo de la imagen y diferenciarlo del de las figuras se planteó la idea de inspeccionar solo sus esquinas, pero como la ubicación de las figuras era arbitraria se presentaba el caso en el que se terminaba inspeccionando el color de una figura.

Solución: Se recorre todo el contorno de la imagen y se almacenan los diversos colores obtenidos, al final el que aparece con más frecuencia es denominado el color del fondo.

Vértices de más en la simplificación mediante algoritmo RDP

Descripción: Cuando el algoritmo RDP recorre la lista de puntos (el contorno), no necesariamente comienza en una esquina, por lo que el punto en el que inicia es considerado como un vértice más, resultando en un cálculo erróneo del número de vértices.

Solución: Simplemente se hace un decremento en el número de vértices calculados por el algoritmo: $\text{numVertices} = \text{numVertices} - 1$; Así el error de cálculo queda resuelto en una sola línea sin modificar el algoritmo original.

Píxeles de colores distintos en las orillas de la figura

Descripción: Algunos píxeles en la orilla de la figura pueden tener un ligero cambio de color cuando se aproximan al fondo.

Solución: Filtrar componentes si son diferentes al fondo.

Factor de circularidad aumentado

Descripción: Al trabajar con píxeles, no se tiene la misma precisión de un cálculo matemático, todas las figuras muestran un aumento en su factor de circularidad y algunas incluso se clasifican como círculos.

Solución: Aumentar el umbral de circularidad y hacer una comparación para clasificarlo.

3.3 Requisitos funcionales

1. **Lectura de imagen:** El sistema debe recibir la ruta de una imagen .bmp y cargarla en la memoria para procesarlo.
2. **Detección de fondo:** El sistema debe poder reconocer el color del fondo para no considerarlo una figura.
3. **Separación de figuras:** El sistema debe ser capaz de aislar cada figura mediante un algoritmo de exploración.
4. **Cálculo de métricas:** El programa debe calcular tanto el área, perímetro, número de vértices y factor de circularidad de cada figura encontrada.

5. **Clasificación de figuras:** El sistema debe de clasificar cada figura encontrada en su categoría correspondiente siguiendo criterios lógicos y utilizando las métricas calculadas.

3.4 Requisitos no funcionales:

1. **Eficiencia:** El sistema debe tener una complejidad aceptable para cada uno de sus algoritmos, se debe tener una complejidad lineal $O(N \times M)$, respecto al ancho y alto de la imagen para procesarlas en pocos segundos.
2. **Tolerancia a fallas:** El programa debe ser capaz de soportar entradas inválidas y no detenerse
3. **Amigabilidad:** El sistema debe ser fácil de comprender para el usuario, permitiendo un uso intuitivo del mismo.
4. **Escalabilidad:** El sistema debe ser capaz de manejar una mayor cantidad de trabajo o datos sin perder velocidad ni rendimiento.
5. **Portabilidad:** El sistema debe ser capaz de ejecutarse en cualquier sistema operativo.

4. Selección de la mejor alternativa

Como se menciona en la sección 2.1 Lenguaje utilizado, no utilizamos un solo algoritmo para la detección y clasificación de figuras, si no que trabajamos de manera modular dividiendo el proyecto en 2 módulos que cargan la responsabilidad por separado:

Módulo 1: Se encarga de la lectura y segmentación de la imagen .bpm para convertirla en estructuras de datos manipulables.

Módulo 2: Toma los datos creados por el módulo 1 y con ellos calcula las métricas necesarias para la clasificación de las figuras.

En el módulo 1 se eligió un algoritmo de búsqueda bfs porque procesa los píxeles nivel por nivel de forma homogénea, además que evita el riesgo de desbordamiento de pila.

En el módulo 2 se eligió el algoritmo Ramer-Douglas-Peucker (RDP) para la simplificación de contornos por diversas razones:

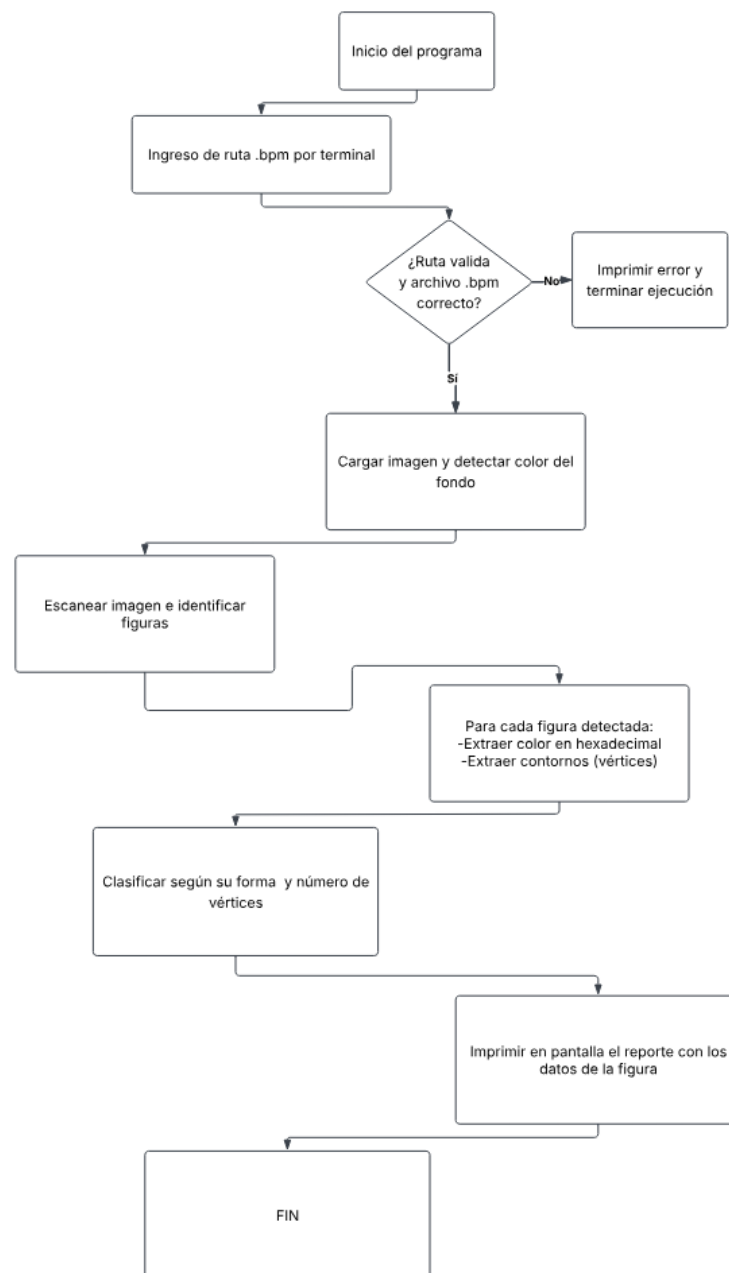
- Simplifica el contorno y extrae solo los vértices utilizando un umbral de tolerancia "Epsilon" (ϵ), este umbral se puede modificar por lo que nos permitía observar como se comportaba la simplificación con un epsilon menor o mayor y así llegar a una simplificación más precisa.
- La simplificación de contorno anteriormente mencionada opera en un tiempo promedio de $O(N \log N)$, $O(n^2)$ en el peor de los casos.
- La transformada de hough era otra alternativa, pero tiene un costo computacional alto porque se evalúa cada píxel de borde a través de un espacio de parámetros multidimensional.

¿Por qué elegimos trabajar de forma modular y no con un solo algoritmo?

Trabajar de forma modular no solo divide la carga de trabajo de una manera más clara y eficaz, sino que también permite que el sistema tenga una estructura “descendente”, donde las clases del módulo 1 se encargan exclusivamente de una tarea y su resultado es el objeto de trabajo de las clases del módulo 2 que finalmente clasifica las figuras.

Así, cada clase tiene sólo una responsabilidad y si el programa necesita modificarse después no es necesario complicarse modificando todo un algoritmo, solo se evalúa el cambio que se quiere hacer y se modifica la clase necesaria.

5. Diagrama de flujo



El diagrama de flujo constituye una representación esquemática del pipeline de procesamiento completo implementado en el sistema.